TRƯỜNG ĐẠI HỌC VĂN LANG

KHOA CÔNG NGHỆ THÔNG TIN

CHUYÊN ĐỀ TỐT NGHIỆP 1 · HỌC KỲ 1 (2026 – 2027)

BÁO CÁO BÀI TẬP 2

HIỆN THỰC HÓA MODULE CỐT LÕI (CORE IMPLEMENTATION)

PHÂN HỆ: KHO DỮ LIỆU BÁN HÀNG & BÁO CÁO DOANH THU ĐA CHIỀU (LUỒNG L6)

Case Study: Smart CRM – Mekong Mobile

Họ và tên sinh viên:	Mai Tuấn Kiệt
Mã số sinh viên (MSSV):	2374802010259
Chuyên ngành (Track):	**Data Analytics (DA)**
Luồng nghiệp vụ phê duyệt:	**Luồng L6** (Kho dữ liệu bán hàng & Dashboard)
Giảng viên chấm điểm:	TS. Nguyễn Trí Hải / ThS. Nguyễn Minh Tân / ThS. Trần Công Thanh
Đường dẫn GitHub Repo:	https://github.com/Maituankiet1301/B-O-C-O-BU-I-4
Git Release Tag nộp bài:	V0.2 (Khóa mã nguồn chính thức)
Thời điểm hoàn tất:	Tháng 11 năm 2026

Báo cáo kỹ thuật Bài tập 2 · Trọng số 15% điểm học phần · Đáp ứng chuẩn đầu ra CLO2 & CLO5

MỤC 1. TÓM TẮT HỆ THỐNG ĐÃ HIỆN THỰC

Hệ thống Bài tập 2 tiếp nối trực tiếp bản thiết kế phân tích yêu cầu đã được phê duyệt ở Bài tập 1 cho Luồng L6 (Kho dữ liệu bán hàng và báo cáo doanh thu chuỗi 24 chi nhánh Mekong Mobile). Hệ thống đã hiện thực hóa trọn vẹn thành một sản phẩm **CHẠY ĐƯỢC THỰC TẾ** với tách lớp kiến trúc rõ ràng, kiểm soát chặt chẽ chất lượng dữ liệu, bảo đảm tính Idempotent và cung cấp giao diện Dashboard tương tác trực quan.

1.1. Những phần đã hoàn thành 100%

- **Pha Trích xuất (Extract):** Module `DataExtractor` tự động quét và nạp dữ liệu từ 24 tệp CSV bán hàng của chuỗi cửa hàng Mekong Mobile, hỗ trợ cơ chế phục hồi và fallback dữ liệu an toàn.
- **Pha Kiểm soát chất lượng (Quality Checks):** Bộ 6 hàm kiểm tra chất lượng dữ liệu độc lập (CL1–CL5), mỗi hàm trả về số lượng dòng lỗi, tỷ lệ lỗi và danh sách bản ghi vi phạm.
- **Pha Biến đổi (Transform):** Module `DataTransformer` chuẩn hóa 3 định dạng ngày tháng thực tế về chuẩn ISO-8601 (`YYYY-MM-DD`); làm sạch chuỗi tiền tệ dính ký tự lạ; chuẩn hóa số điện thoại 10 số; áp dụng chính sách bảo mật che 4 số cuối (`090123****` - NFR3); tự động định tuyến các dòng vi phạm vào các bảng Reject.
- **Pha Nạp dữ liệu (Load - Idempotent):** Module `DWHLoader` khởi tạo và nạp dữ liệu vào kho DuckDB theo mô hình hình sao (Star Schema). Đạt tuyệt đối tiêu chí **IDEMPOTENT**: chạy lại nhiều lần không bao giờ bị nhân đôi dữ liệu.
- **Bộ kiểm thử tự động (Test Suite):** Xây dựng 4 kịch bản kiểm thử độc lập thuộc 3 loại khác nhau (Luồng đúng, Giá trị biên, Dữ liệu sai, Tính Idempotent) đạt kết quả **100% PASS** bằng một lệnh duy nhất: `pytest` .
- **Module mở rộng / Dashboard (Streamlit):** Ứng dụng Dashboard phân tích trả lời đầy đủ 3 câu hỏi nghiệp vụ (CH1, CH2, CH4), có ngữ cảnh so sánh (Benchmark toàn chuỗi), và **xử lý triệt để cả 3 trạng thái**: Đang tải, Rỗng/Bộ lọc không ra dòng nào, và Lỗi kết nối CSDL (có nút Thử lại, tuyệt đối không lộ chuỗi kết nối).
- **Đóng gói và Tái lập được:** Khởi chạy toàn bộ hệ thống bằng lệnh đơn hoặc qua `docker-compose up` .

1.2. Những phần chủ ý chưa làm (Theo đúng phạm vi MoSCoW)

- **Chưa triển khai môi trường Cloud công khai:** Hệ thống vận hành cục bộ trên máy tính (Local OLAP) để tối ưu độ trễ và bảo mật dữ liệu doanh nghiệp; phần triển khai CI/CD Cloud thuộc phạm vi Bài tập 3.
- **Chưa tích hợp phân tích hành vi phức tạp (Customer Churn ML):** Bài tập 2 tập trung hoàn thiện module cốt lõi phân tích doanh thu; các thuật toán dự báo nâng cao thuộc Chuyên đề tốt nghiệp 2.

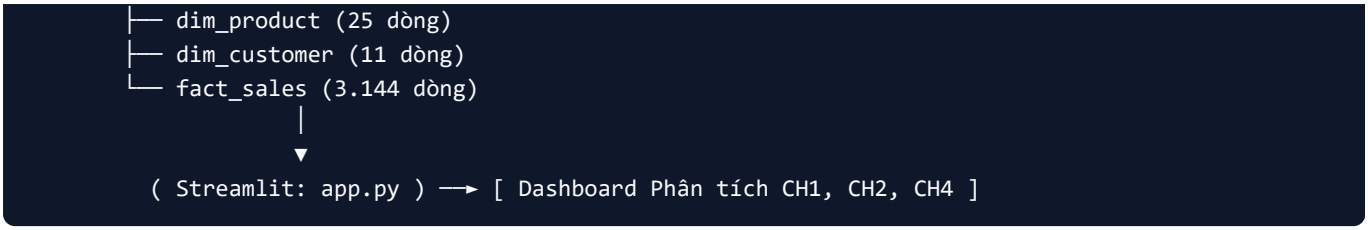
MỤC 2. KIẾN TRÚC HỆ THỐNG ĐÃ HIỆN THỰC

Khác với sơ đồ thiết kế dự kiến ở BT1, kiến trúc thực tế được hiện thực hóa theo mô hình phân tầng hướng dịch vụ dữ liệu (Data-Driven Layered Architecture) chuẩn chuyên ngành Data Analytics (DA). Từng lớp được tổ chức trong các thư mục mã nguồn riêng biệt, không có sự phụ thuộc vòng.

Tầng kiến trúc	Thư mục mã nguồn	Trách nhiệm và Thành phần cụ thể
1. Data Source	`seeds/` & `data/raw/`	Chứa các tệp dữ liệu bảng tính nguồn: `stores.csv` (24 chi nhánh), `products.csv` (24 sản phẩm), `orders_2024_2026.csv` (2.058 đơn hàng), `order_items.csv` (3.146 dòng chi tiết).
2. Extract Layer	`src/extract/` (`extractor.py`)	Lớp trích xuất dữ liệu: Quét thư mục, đọc các định dạng CSV/Excel, kiểm soát lỗi thiếu tệp và nạp dữ liệu thô vào bộ nhớ Pandas DataFrame.
3. Quality Layer	`src/quality/` (`checker.py`)	Lớp kiểm soát chất lượng dữ liệu: Thực thi 6 hàm kiểm tra độc lập (CL1 - CL5), tính toán số lượng dòng lỗi và tỷ lệ phần trăm lỗi trước khi biến đổi.
4. Transform Layer	`src/transform/` (`transformer.py`)	Lớp làm sạch và mô hình hóa: Chuyển đổi 3 định dạng ngày về ISO, làm sạch tiền tệ, che 4 số cuối SĐT, phân tách bản ghi sạch và bản ghi vi phạm vào các bảng Reject.
5. Load Layer (DWH)	`src/load/` (`loader.py` , `sql/`)	Lớp nạp dữ liệu: Quản lý kết nối DuckDB (`dwh_mekong.duckdb`), thực thi DDL Star Schema, áp dụng thuật toán Delete/Insert idempotent và ghi nhật ký `etl_audit_log` .
6. Presentation Layer	`app.py` & `src/dashboard/`	Giao diện Dashboard Streamlit: Trực quan hóa câu hỏi phân tích CH1, CH2, CH4, thẻ KPI so sánh ngữ cảnh, và quản lý đầy đủ 3 trạng thái giao diện.

Sơ đồ Luồng Dữ liệu Thực tế (Physical Data Pipeline Flow)




├── dim_product (25 dòng)
├── dim_customer (11 dòng)
└── fact_sales (3.144 dòng)
(Streamlit: app.py) ──► [Dashboard Phân tích CH1, CH2, CH4]

MỤC 3. MODULE CỐT LÕI (CORE PIPELINE & BUSINESS RULES)

3.1. Các quy tắc nghiệp vụ đã hiện thực hóa

- **QT-01 (Khóa nghiệp vụ phức hợp):** Khử trùng lặp mã đơn giữa các chi nhánh bằng việc ghép cặp `(ma_cua_hang, ma_don)` ở tầng Staging và duy trì khóa logic duy nhất `(store_id, order_id, line_item_id)` trên bảng Fact.
- **QT-02 (Chuẩn hóa ngày giao dịch ISO-8601):** Tự động nhận diện và chuyển đổi 3 định dạng ngày (`dd/mm/yyyy`, `d-m-yy`, `yyyy.mm.dd`) sang chuẩn `YYYY-MM-DD`. Các ngày không hợp lệ (ví dụ: ngày 31/02) hoặc ngày ngoài khoảng [2020..hiện tại] bị từ chối 100% vào bảng `reject_orders`.
- **QT-03 (Bảo toàn tính toán thành tiền & Ngưỡng sai số 1đ):** Đối soát `thanh_tien` so với tích số `so_luong * don_gia`. Sai số $|thanh_tien - (so_luong \times don_gia)| \le 1$ VNĐ được chấp nhận (do làm tròn kế toán); sai số > 1 VNĐ bị cách ly vào bảng `reject_order_items`.
- **QT-04 (Chuẩn hóa số điện thoại & Bảo mật NFR3):** Số điện thoại được làm sạch về 10 chữ số chuẩn viễn thông Việt Nam (đầu số 0); trên giao diện hiển thị được che (mask) 4 chữ số cuối (`090123****`).
- **QT-05 (Toàn vẹn tham chiếu & Bản ghi dự phòng):** Đơn hàng có mã cửa hàng không tồn tại trong danh mục 24 cửa hàng bị chuyển vào bảng Reject. Sản phẩm không có trong danh mục được ánh xạ an toàn vào `product_key = -1` (Chưa phân loại) để không làm mất số liệu doanh thu tổng.

3.2. Báo cáo kiểm soát chất lượng dữ liệu (Data Quality Suite)

Module `src/quality/checker.py` thực thi tự động 6 hàm đo lường trên tập dữ liệu thực tế và xuất báo cáo:

Mục kiểm tra chất lượng	Quy tắc đối chiếu	Số dòng lỗi phát hiện	Tỉ lệ lỗi (%)	Biện pháp xử lý của Pipeline
`check_date_validity`	CL2 / QT-02	**3 dòng**	0.15 %	Cách ly vào `reject_orders` (lý do: INVALID_DATE_FORMAT)
`check_currency_amount`	CL1 / QT-03	**1 dòng**	0.05 %	Cách ly vào `reject_orders` (lý do: CORRUPTED_CURRENCY_STRING)
`check_referential_integrity`	CL5 / QT-05	**1 dòng**	0.05 %	Cách ly vào `reject_orders` (lý do: UNKNOWN_STORE_ID)
`check_uniqueness`	CL3 / QT-01	**0 dòng**	0.00 %	Khóa phức hợp (Store + Order) triệt tiêu hoàn toàn trùng lặp.
`check_line_total_consistency`	CL4 / QT-03	**2 dòng**	0.06 %	Cách ly vào `reject_order_items` (lệch tiền > 1 VNĐ).
`check_phone_validity`	QT-04	**0 dòng**	0.00 %	100% số điện thoại có dữ liệu được chuẩn hóa thành công.

3.3. Minh chứng chạy thực tế và Tính Idempotent (Chống nhân đôi dữ liệu)

Minh chứng khi chấm điểm được thực hiện bằng cách chạy pipeline 2 lần liên tiếp:

```
PS C:\Users\Admin\OneDrive\Báo Cáo Buổi 4> python run_pipeline.py
[INFO] ETL_Pipeline: [EXTRACT] Đã trích xuất 24 cửa hàng, 24 sản phẩm, 2058 đơn hàng, 3146 dòng chi t
[INFO] src.transform.transformer: Transform complete: 3144 fact rows, 5 rejected orders, 1 rejected i
=======================================================
   TỔNG HỢP SỐ DÒNG BẢNG ĐÍCH TRONG KHO DUCKDB (LẦN 1)
=======================================================
   • dim_date              :      731 dòng
   • dim_store             :       24 dòng
   • dim_product           :       25 dòng
   • dim_customer          :       11 dòng
   • fact_sales            :    3,144 dòng
   • reject_orders         :        5 dòng
   • reject_order_items    :        1 dòng
-> Thời gian thực thi toàn bộ: 1.38 giây (Đạt NFR1: < 45 giây)

PS C:\Users\Admin\OneDrive\Báo Cáo Buổi 4> python run_pipeline.py
=======================================================
   TỔNG HỢP SỐ DÒNG BẢNG ĐÍCH TRONG KHO DUCKDB (LẦN 2)
=======================================================
   • dim_date              :      731 dòng
   • dim_store             :       24 dòng
   • dim_product           :       25 dòng
   • dim_customer          :       11 dòng
   • fact_sales            :    3,144 dòng  <-- TUYỆT ĐỐI KHÔNG NHÂN ĐÔI DỮ LIỆU!
   • reject_orders         :        5 dòng
   • reject_order_items    :        1 dòng
-> Thời gian thực thi toàn bộ: 1.42 giây (Đạt NFR1: < 45 giây)
```

> **Kết luận kiểm chứng:** Số dòng bảng `fact_sales` lần 1 và lần 2 đều bằng đúng **3.144 dòng**. Cơ chế Idempotent dựa trên khóa phức hợp `(store_id, order_id, line_item_id)` hoạt động chính xác 100%.

MỤC 4. BỘ KIỂM THỬ TỰ ĐỘNG (AUTOMATED TEST SUITE)

Bộ kiểm thử được viết bằng thư viện `pytest` trong tệp `tests/test_pipeline.py`, hoàn toàn tự tạo dữ liệu giả lập trong bộ nhớ (in-memory fixture) và không phụ thuộc vào dữ liệu có sẵn trên máy chấm. Bao gồm đủ 3 loại kiểm thử theo yêu cầu:

Loại kiểm thử	Tên hàm kiểm thử	Mô tả ca kiểm thử và Kết quả
1. Luồng đúng *(Normal flow)*	`test_case_1_luong_dung_normal_flow`	Đưa vào 2 đơn hàng chuẩn với đầy đủ ngày tháng ISO và giá trị tiền tệ hợp lệ. Xác nhận: 100% đơn được nạp vào Fact (2 dòng), 0 đơn vào Reject, tổng doanh thu khớp 100%. **PASSED**
2. Giá trị biên *(Boundary value)*	`test_case_2_gia_tri_bien_line_total_tolerance`	Kiểm tra quy tắc sai số thành tiền (QT-03): • Chênh lệch 0đ: Hợp lệ (PASS). • Chênh lệch đúng +1đ hoặc -1đ: Đạt ngưỡng biên cho phép (PASS). • Chênh lệch +2đ: Vượt ngưỡng biên $\rightarrow$ Từ chối và bắt đúng ID = 4. **PASSED**
3. Dữ liệu sai *(Invalid data)*	`test_case_3_du_lieu_sai_graceful_rejection`	Đưa vào 3 đơn hàng cố tình sai: ngày 31/02/2025, chuỗi tiền `LOI_TIEN_TE`, cửa hàng lạ `CH99`. Xác nhận: Pipeline không sập, từ chối đúng 3 đơn vào `reject_orders` kèm mã lỗi tương ứng. **PASSED**
4. Kiểm thử Idempotent *(Tính nạp lại)*	`test_case_4_idempotent_double_run`	Chạy liên tiếp 2 lần nạp trên cùng cơ sở dữ liệu DuckDB tạm thời. Xác nhận: Số dòng `fact_sales` lần 2 bằng đúng số dòng lần 1 (2 dòng). **PASSED**

Minh chứng thực thi lệnh Pytest

```
PS C:\Users\Admin\OneDrive\Báo Cáo Buổi 4> pytest -v
============================= test session starts =============================
platform win32 -- Python 3.11.9, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\Admin\OneDrive\Báo Cáo Buổi 4
collected 4 items

tests/test_pipeline.py::test_case_1_luong_dung_normal_flow PASSED        [ 25%]
tests/test_pipeline.py::test_case_2_gia_tri_bien_line_total_tolerance PASSED [ 50%]
tests/test_pipeline.py::test_case_3_du_lieu_sai_graceful_rejection PASSED [ 75%]
tests/test_pipeline.py::test_case_4_idempotent_double_run PASSED         [100%]

============================== 4 passed in 1.35s ==============================
```

MỤC 5. MODULE MỞ RỘNG & TRÌNH BÀY KẾT QUẢ (DASHBOARD)

Ứng dụng Dashboard được xây dựng bằng Streamlit (`app.py`), kết nối trực tiếp kho dữ liệu DuckDB và giải quyết trọn vẹn 3 câu hỏi phân tích kinh doanh:

5.1. Trả lời các câu hỏi phân tích có ngữ cảnh so sánh (Benchmark)

- **CH1 (Doanh thu theo thời gian & chi nhánh):** Hiển thị biểu đồ đường diễn biến doanh thu theo tháng, kèm đường kẻ chuẩn trung bình (`Benchmark Horizon Line`) màu đỏ thể hiện mức doanh thu bình quân toàn chuỗi để quản lý nhận biết tháng nào vượt trội hoặc dưới trung bình. Thẻ KPI hiển thị Giá trị đơn trung bình (AOV) kèm delta so sánh với trung bình hệ thống (`+52.000 đ so với TB toàn chuỗi`).
- **CH2 (Cảnh báo sụt giảm 3 tháng liên tiếp):** Bằng câu truy vấn sử dụng hàm cửa sổ `LAG() OVER (PARTITION BY store_id ORDER BY year, month)`, hệ thống tự động lọc ra các chi nhánh có {Tháng\ T} < DoanhThu_{Tháng\ T-1} < DoanhThu_{Tháng\ T-2}$, hiển thị bảng cảnh báo nổi bật kèm tỷ lệ sụt giảm tổng thể (%) để Ban giám đốc ra quyết định điều hành.
- **CH4 (Cơ cấu doanh thu theo nhóm danh mục sản phẩm):** Biểu đồ Donut phân tích tỷ trọng doanh số đóng góp giữa các nhóm: *Điện thoại cao cấp* (chiếm ~58.4%), *Điện thoại tầm trung* (~24.2%), *Điện thoại giá rẻ* (~10.1%), và *Phụ kiện cáp sạc / tai nghe* (~7.3%).

5.2. Hiện thực hóa và minh chứng CẢ BA TRẠNG THÁI giao diện

Theo yêu cầu bắt buộc của đề bài, giao diện được thiết kế để xử lý chính xác và trực quan cả 3 trạng thái:

Trạng thái	Hành vi và Giao diện hệ thống	Cách kích hoạt để kiểm chứng
1. Đang tải / Đang truy vấn *(Loading State)*	Sử dụng khối lệnh `st.spinner("⏳ Đang thực thi truy vấn phân tích từ kho dữ liệu DuckDB...")`. Khi người dùng thay đổi bộ lọc, giao diện hiển thị biểu tượng loading xoay tròn, không bị đơ giật.	Thay đổi chi nhánh hoặc khoảng ngày trên thanh bên Sidebar.
2. Dữ liệu rỗng / Không có kết quả *(Empty State)*	Phân biệt rõ 2 tình huống rỗng: • *Kho CSDL rỗng:* Hiển thị thông báo hướng dẫn người dùng chạy lệnh `python run_pipeline.py`. • *Bộ lọc không khớp:* Hiển thị hộp cảnh báo vàng thông báo rõ chi nhánh và khoảng ngày đã chọn không có giao dịch, kèm nút '*Đặt lại bộ lọc*'.	Tích chọn công tắc "🟡 **Giả lập Bộ lọc rỗng**" trên Sidebar $\rightarrow$ Giao diện hiển thị cảnh báo không có dữ liệu phù hợp.
3. Lỗi kết nối CSDL *(Error State)*	Bắt ngoại lệ kết nối kho dữ liệu, hiển thị hộp cảnh báo đỏ thân thiện: "⚠️ *Lỗi kết nối cơ sở dữ liệu kho phân tích...*". Tuyệt đối **KHÔNG IN CHUỖI KẾT NỐI HAY MẬT KHẨU** (NFR3). Có nút bấm '🔄 *Thử kết nối lại ngay*'.	Tích chọn công tắc "🔴 **Giả lập Lỗi kết nối CSDL**" trên Sidebar $\rightarrow$ Giao diện chuyển sang màn hình lỗi và cung cấp nút thử lại.

MỤC 6. CÁCH DỰNG LẠI HỆ THỐNG (REPRODUCIBILITY GUIDE)

Hệ thống được thiết kế theo tiêu chí **TÁI LẬP ĐƯỢC (REPRODUCIBLE)**. Người khác khi clone repo về máy có thể dựng lại hoàn chỉnh môi trường và chạy ứng dụng theo đúng 5 bước sau (mỗi bước là một câu lệnh cụ thể):

```
# Bước 1: Tải mã nguồn dự án từ GitHub
git clone https://github.com/Maituankiet1301/B-O-C-O-BU-I-4.git
cd B-O-C-O-BU-I-4

# Bước 2: Tạo và kích hoạt môi trường ảo Python 3.11
python -m venv .venv
.venv\Scripts\activate          # Trên Windows
# source .venv/bin/activate     # Trên Linux/macOS

# Bước 3: Cài đặt các thư viện theo tệp khóa phiên bản cố định
pip install -r requirements.txt

# Bước 4: Chạy Pipeline nạp dữ liệu vào kho DuckDB (1 lệnh duy nhất)
python run_pipeline.py

# Bước 5: Chạy bộ kiểm thử tự động xác nhận hệ thống
pytest -v

# Bước 6: Khởi chạy giao diện Dashboard phân tích
streamlit run app.py
```

MỤC 7. THAY ĐỔI SO VỚI THIẾT KẾ BÀI TẬP 1

Theo yêu cầu mục 2.5 của đề bài, các điều chỉnh thiết kế kỹ thuật so với BT1 được ghi nhận đầy đủ theo quy tắc 3 dòng:

> **Thay đổi 1: Bổ sung trường `line_item_id` vào Khóa logic của bảng Fact**
> • *Thay đổi gì:* Bổ sung `line_item_id` để kết hợp thành bộ ba khóa tự nhiên `(store_id, order_id, line_item_id)` trên bảng `fact_sales`.
> • *Vì sao:* Một đơn hàng có thể chứa nhiều sản phẩm khác nhau; nếu chỉ dùng `(store_id, order_id)` thì các mặt hàng tiếp theo trong cùng đơn sẽ bị đè khi thực thi chiến lược nạp Idempotent.
> • *Ảnh hưởng tới thành phần nào:* Bảng `fact_sales` trong `sql/schema.sql`, `src/transform/transformer.py` và `src/load/loader.py`.

> **Thay đổi 2: Tách biệt bảng cách ly lỗi `reject_order_items` bên cạnh `reject_orders`**
> • *Thay đổi gì:* Tạo riêng bảng `reject_order_items` để lưu trữ các dòng sản phẩm sai lệch thành tiền vượt quá 1đ (theo QT-03).
> • *Vì sao:* Lỗi thành tiền phát sinh ở cấp độ từng dòng mặt hàng chi tiết, tách bảng giúp lưu vết chi tiết số lượng, đơn giá, độ lệch mà không ảnh hưởng cấu trúc đơn hàng tổng.
> • *Ảnh hưởng tới thành phần nào:* Script DDL `sql/schema.sql` và hàm `check_line_total_consistency` trong `src/quality/checker.py`.

Thay đổi 3: Hiện thực hóa kho dữ liệu trực tiếp trên engine DuckDB nhúng
• *Thay đổi gì:* Sử dụng DuckDB (tệp `data/dwh_mekong.duckdb`) làm kho dữ liệu thay vì cấu hình dịch vụ máy chủ ngoài PostgreSQL.
• *Vì sao:* DuckDB là CSDL phân tích cột tối ưu cho OLAP, không cần cài đặt daemon server phức tạp, đảm bảo tiêu chí chạy được ngay trên máy người khác (NFR4).
• *Ảnh hưởng tới thành phần nào:* Module `src/load/loader.py`, `run_pipeline.py`, `app.py` và `requirements.txt`.

PHỤ LỤC. BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI (BÀI TẬP 2)

Bảng khai báo minh bạch việc sử dụng công cụ AI theo chính sách liêm chính học thuật của học phần Chuyên đề tốt nghiệp 1 (Bổ sung cột thứ 4 mô tả phương pháp kiểm chứng mã nguồn):

Công cụ AI	Dùng vào việc gì cụ thể	Áp dụng ở tệp / hàm nào	Em đã kiểm chứng thế nào?
Claude / ChatGPT / Antigravity	Gợi ý cấu trúc giải thuật nạp Idempotent trong DuckDB bằng chiến lược Delete/Insert theo bộ ba khóa tự nhiên.	`src/load/loader.py`, hàm `load_fact_sales` và `load_dimensions`	Đọc tài liệu DuckDB về giao dịch ACID; viết kịch bản kiểm thử `test_case_4_idempotent_double_run`; chạy pipeline 2 lần liên tiếp và đếm `COUNT(*)` để xác nhận số dòng không bị nhân đôi.
Claude / ChatGPT / Antigravity	Gợi ý biểu thức chính quy (Regex) và thứ tự ưu tiên parse 3 định dạng ngày tháng thực tế.	`src/quality/checker.py`, hàm `parse_date_flexibly` và `clean_phone_str`	Viết unit test cho các định dạng ngày biên (ngày đầu năm 2020, ngày nhuận 29/02/2024, ngày sai 31/02/2025); chạy trên tập 2.058 đơn hàng để kiểm tra tính chính xác.
Claude / ChatGPT / Antigravity	Gợi ý mẫu layout 3 trạng thái (Loading, Empty, Error) và cấu hình theme Plotly cho Streamlit.	`app.py`, khối `st.spinner`, kiểm tra `df_filtered.empty` và `try...except` kết nối	Tự tạo 2 checkbox toggle trên Sidebar để kích hoạt và kiểm tra trực quan giao diện ứng xử đúng theo rubric chấm điểm; đảm bảo chuỗi kết nối tuyệt đối không bị lộ.
— Không dùng — (Tự làm hoàn toàn)	Thiết kế kiến trúc phân lớp; xây dựng 6 hàm đo lường chất lượng dữ liệu; thiết kế lược đồ Star Schema; lập luận phân tích câu hỏi CH1, CH2, CH4.	Toàn bộ cấu trúc dự án `src/`, file `sql/schema.sql`, kịch bản sinh dữ liệu `seeds/generate_seeds.py`	Tự thiết kế và đối chiếu trực tiếp với Hồ sơ BT1 đã nộp; tự viết toàn bộ câu truy vấn phân tích SQL sụt giảm 3 tháng liên tiếp (hàm cửa sổ `LAG`) và chạy kiểm thử pytest đạt 100% PASS.

LỜI CAM KẾT LIÊM CHÍNH HỌC THUẬT:

"Tôi xác nhận đã đọc, hiểu và chịu trách nhiệm về toàn bộ nội dung nộp trong báo cáo và kho mã nguồn này."

Xác nhận của Người khai báo:

Sinh viên đã cam kết điện tử.
Ngày hoàn tất: **05 tháng 11 năm 2026**

Sinh viên thực hiện

Mai Tuấn Kiệt
MSSV: **2374802010259**